

SPLITLY

Software Architecture and Design

Responsive Web Application

Modular Monolith

Event-Driven Notifications

Gemini 2.5 Flash API

ARCHITECTURE BASELINE AND MVP TARGET

16 July 2026

Table of Contents

1. Executive Summary	4
2. Goals, Scope, and Architecture Drivers	4
2.1 MVP goals	4
2.2 In scope	4
2.3 Out of scope for the MVP	4
2.4 Quality priorities	5
3. Architecture Decisions	5
3.1 Technology stack baseline	6
4. System Context	6
Trust boundaries	6
5. Container and Deployment View	7
6. Backend Module Design	7
Dependency rules	8
Notification event model	8
7. Core Runtime Flows	9
7.1 Scan, verify, and create a bill	9
7.2 Item allocation and charge distribution	9
7.3 Settle up	10
8. Data Design	10
Data consistency	11
9. Receipt Image Lifecycle and Privacy	11
10. External Integration Contracts and Fallbacks	12
11. API Design	12
Contract conventions	12
High-risk route review before production	12
12. Security Architecture	13
Authentication and session	13
Authorization	13
Application and infrastructure	13

13. Reliability, Performance, and Observability	13
Initial service targets	13
Observability	14
14. UX Routes and Architecture Traceability	14
15. Technical POC and Delivery Plan	15
POC-1: Core calculation (acceptance gate)	15
POC-2: OCR-assisted entry	15
POC-3: Settlement assistance	15
Recommended delivery order	15
16. Risks, Constraints, and Mitigations	16

1. Executive Summary

Splitly is a responsive web application that turns a receipt into a transparent settlement: scan or enter a bill, verify the extracted data, assign items to participants, distribute tax/fees/discounts, calculate each share, and show who owes whom.

The MVP uses a **modular monolith** rather than microservices. A React single-page application communicates with one Node.js/Express API, which organizes business capabilities into routes, controllers, services, and MongoDB models. MongoDB stores users, groups, bills, item allocations, payment state, activities, notifications, and transactional outbox events. Gemini 2.5 Flash, VietQR, and email are external adapters at the system boundary. Domain events and an asynchronous notification worker decouple bill/payment transactions from notification delivery; MongoDB remains the source of truth.

This is the best fit for the current team and product maturity: one backend codebase keeps transactions, debugging, testing, and operating cost manageable, while the API and worker processes can be scaled independently when measured load requires it.

2. Goals, Scope, and Architecture Drivers

2.1 MVP goals

1. Complete the simplest reliable flow: create group -> enter/scan bill -> verify -> assign items -> calculate shares -> show debt -> track payment.
2. Make every monetary result explainable and preserve the invariant that allocated shares equal the bill total.
3. Keep the product usable when OCR, VietQR, event delivery, or email is unavailable.
4. Protect receipt contents, identity data, bank details, and group financial data.
5. Deliver a usable mobile and desktop experience without maintaining two native applications.

2.2 In scope

- Email registration, verification, login, logout, and profile/bank-detail management.
- Group and member management.
- Manual, equal, per-person, and item-based bill entry.
- Receipt image validation, resizing, Gemini 2.5 Flash extraction, and human correction.
- Shared-item assignment and proportional distribution of the difference between item subtotal and final total.
- Debt summary, payment reminders, confirmation, and paid/unpaid status.
- VietQR image generation, activity history, reports, email, and event-driven notifications.

2.3 Out of scope for the MVP

- Native iOS/Android applications and offline synchronization.
- Direct bank transfer initiation, bank-webhook reconciliation, or custody of funds.
- Microservices, event sourcing, Kafka, and Elasticsearch.
- Multi-currency conversion and accounting-grade ledger features.
- Long-term storage of receipt images.
- Automatic acceptance of OCR results without user verification.

2.4 Quality priorities

Priority	Architectural consequence
Correctness and transparency	Server-side calculation is authoritative; inputs and totals are validated; rounding remainders are deterministic.
Privacy and security	Least-data receipt handling, authenticated APIs, resource authorization, secure cookies, redacted logs, and encrypted transport.
Availability	Manual entry is always available; failures of optional integrations do not block bill creation or settlement records.
Delivery speed	One frontend and one modular backend, familiar JavaScript stack, managed database, and simple deployment.
Maintainability	Capability-based modules and provider adapters prevent external APIs from leaking into domain logic.

3. Architecture Decisions

Decision	Choice	Rationale and trade-off
Application type	Responsive web application/PWA-ready SPA	One codebase covers phones and desktops and supports camera/file upload. Native-only capabilities and app-store distribution are not necessary for the MVP.
Backend style	Modular monolith	Lowest operational overhead and easiest consistency model for a small team. Module boundaries allow later extraction without paying distributed-system cost now.
Frontend	React 19 + Vite, React Router, Redux Toolkit, MUI/Tailwind	Matches the implemented UI, supports responsive forms and fast iteration. It requires disciplined state ownership to avoid duplicated calculations.
API	Node.js + Express 5 REST API	Matches the implemented service and team language. HTTP controllers remain thin; domain work belongs in services.
Database	MongoDB	Fits bill aggregates with embedded items and payment status and is already implemented. Cross-document workflows need explicit consistency handling and indexes.
Notification delivery	Domain events, transactional outbox, and asynchronous worker	Core transactions commit without waiting for email/in-app delivery. Persisted events support retry, idempotency, and failure recovery.
OCR	Gemini 2.5 Flash API behind a provider adapter	Centralizes credentials and request mapping. OCR output is untrusted draft data and must be parsed, validated, and confirmed by a user.
QR payment	VietQR-generated QR image/instruction	Speeds transfer entry without making Splitly a payment processor. Payment completion still requires confirmation.
Event-driven architecture	Durable domain events via a MongoDB transactional outbox	Bill/payment state and its event commit atomically; a worker claims events and creates notifications with retry and idempotency. A separate broker is unnecessary for MVP throughput.
Event sourcing	Not used	Current-state documents plus append-only activities meet traceability needs. Rebuilding all financial state from events would add schema evolution and operational complexity without MVP value.
Deployment	Separately built web and API on a VPS with PM2; MongoDB external/managed	Reflects the current CI/CD workflows. Docker is recommended for reproducible POC/production packaging but is not present in the current repository.

3.1 Technology stack baseline

Layer	Technology	Role
Web	React 19, Vite 7, React Router 7	Single-page application, routing, responsive user workflows
UI and state	Material UI 7, Tailwind CSS 4, Redux Toolkit, Axios	Components/theme, client state, and HTTP access
API	Node.js 18+, Express 5, Babel	Versioned REST endpoints and application orchestration
Validation/media	Joi, <code>file-type</code> , Sharp	Request schemas, MIME inspection, and receipt resizing
Data	MongoDB Node.js driver 6, MongoDB/Atlas	Canonical state, notifications, and transactional outbox events
Identity	JWT HS256, bcrypt, secure HTTP cookies	Authentication and password protection
Async messaging	MongoDB transactional outbox + Node.js worker	Durable domain-event processing, retry, and notification delivery
AI/OCR	Gemini 2.5 Flash API	Receipt extraction and assistant features
Payment assistance	VietQR image/bank APIs	QR payment instructions; no fund custody
Messaging	SMTP/Nodemailer and Brevo	Verification, reminders, and transactional email
Delivery	GitHub Actions, PM2, VPS	Build and process deployment; containerization remains a target improvement

4. System Context

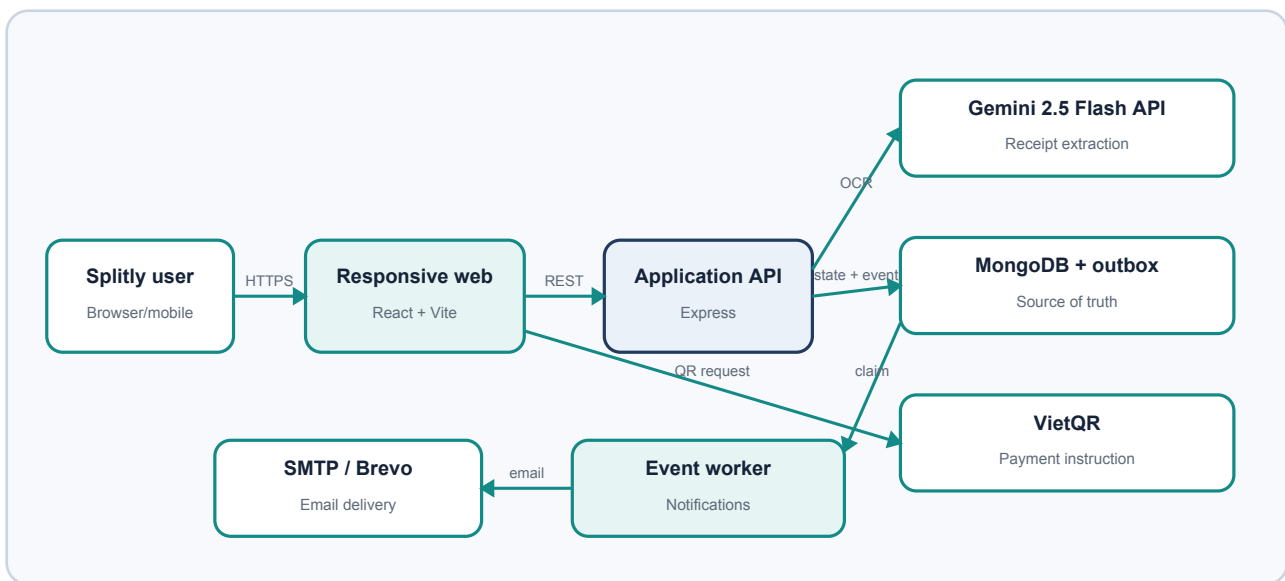


Figure 1. Architecture view

Trust boundaries

- The browser is untrusted. Client-side route guards and calculations improve UX but never replace server validation or authorization.
- The API is the policy enforcement point and owns authoritative calculation and persistence.
- MongoDB is private infrastructure; it is never accessed directly by the browser.
- Gemini, VietQR, and mail services are third parties. Data sent to them is minimized, timed out, and treated as potentially unavailable.

- Splitly creates payment instructions and records confirmations; it does not move money or prove that a bank transfer occurred.

5. Container and Deployment View

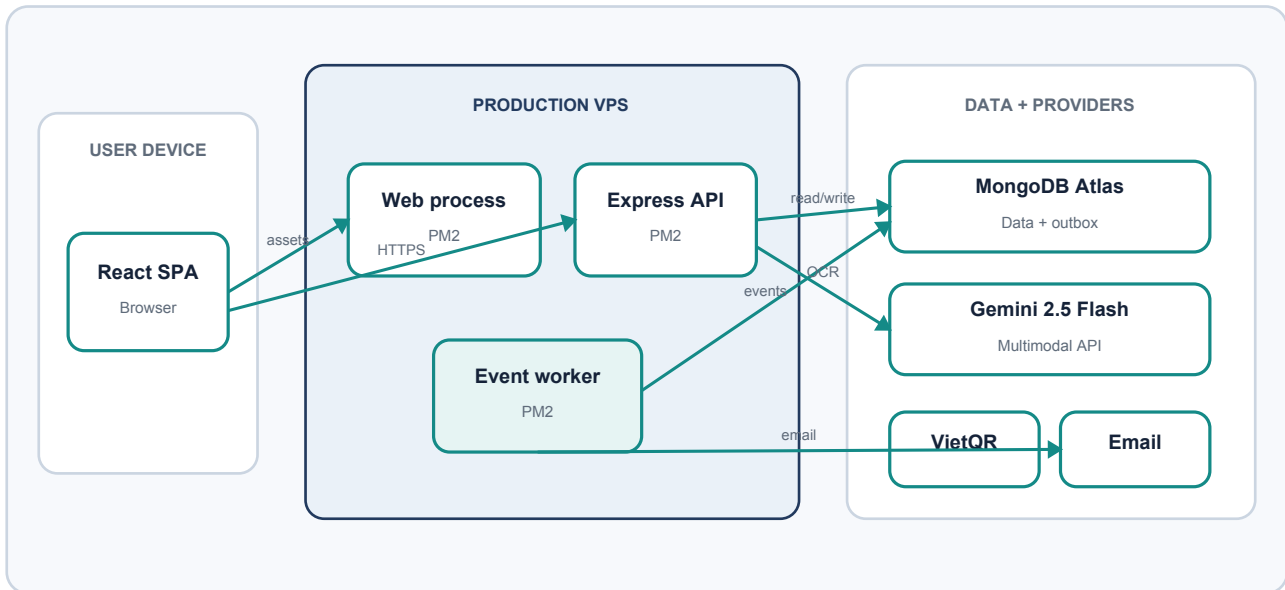


Figure 2. Architecture view

The current GitHub Actions workflows build the API with Babel, build/serve the web application, and restart PM2 processes. The target deployment adds a separately managed worker process from the same backend codebase. Production hardening should put TLS termination/reverse proxy in front of the web/API processes, add independent API and worker health checks, deploy immutable artifacts or containers, and support rollback to the previous build.

6. Backend Module Design

The API is one deployable application with internal layers:

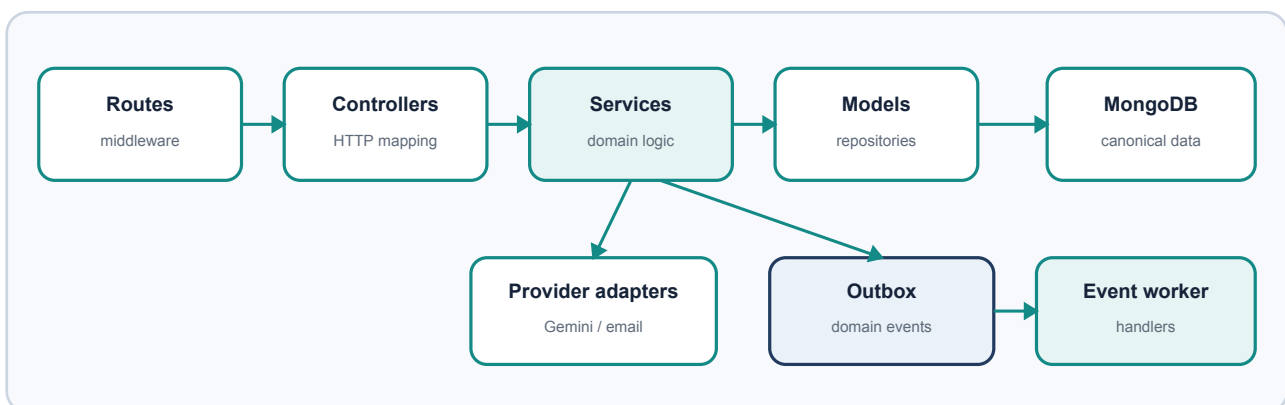


Figure 3. Architecture view

Module	Responsibilities	Primary persisted data
Identity	Registration, verification, login/logout, profile, guest user	users
Groups	Group lifecycle, membership, group/bill association	groups

Module	Responsibilities	Primary persisted data
Bills	Bill validation, split calculation, item assignment, settlement state, opt-out	<code>bills</code>
Debts and payments	Mutual debt view, reminders, payment submission and confirmation	<code>payment_reminders</code> , <code>payment_confirmations</code> , bill payment state
Activity/history	Human-readable audit trail and bill history	<code>activities</code>
Notifications	Event handlers, persisted inbox, email dispatch, retry, and dead-letter handling	<code>notifications</code> , <code>outbox_events</code>
Reporting/dashboard	Derived monthly spending and debt summaries	Reads bills/users; no separate source of truth
AI assistant/OCR	Gemini 2.5 Flash request orchestration and structured receipt draft	No receipt image persistence in the target design
Providers	JWT, Gemini, SMTP/Brevo, and future VietQR server adapter	Secrets in runtime configuration only

Dependency rules

1. Routes select middleware and controllers; they contain no business rules.
2. Controllers translate HTTP input/output and delegate to services.
3. Services own calculations, access-policy decisions, orchestration, and transaction boundaries.
4. Models own schema validation, identifier conversion, indexes, and persistence queries.
5. Provider-specific request/response formats remain behind adapters.
6. Reports and dashboards derive data from canonical bill/payment records and do not mutate them.
7. Services publish domain events by writing to the outbox in the same transaction as business state; workers own side effects and idempotent retries.

Notification event model

Domain event	Produced when	Notification handler result
<code>BillCreated</code>	A verified bill and calculation are committed	Notify non-creator participants and create activity entries
<code>BillUpdated</code>	Bill details, assignments, or deadline change	Notify affected participants with the changed fields
<code>PaymentSubmitted</code>	A debtor submits a payment claim	Notify the creditor to confirm or reject
<code>PaymentConfirmed</code>	A creditor confirms payment	Notify payer and affected participants; refresh settlement state
<code>PaymentRejected</code>	A creditor rejects a payment claim	Notify payer with a safe reason and next action
<code>PaymentReminderRequested</code>	A creditor requests a reminder	Create in-app notification and enqueue email delivery
<code>GroupMemberAdded</code>	A member joins a group	Notify the new member and relevant group members

Delivery is **at least once**. Every outbox event has a unique `eventId`; each handler records that ID or uses a unique derived key so a retry cannot create duplicate notifications or apply a payment twice. Event payloads are versioned and contain references plus minimal display data, never raw receipt images, credentials, or full user records.

7. Core Runtime Flows

7.1 Scan, verify, and create a bill

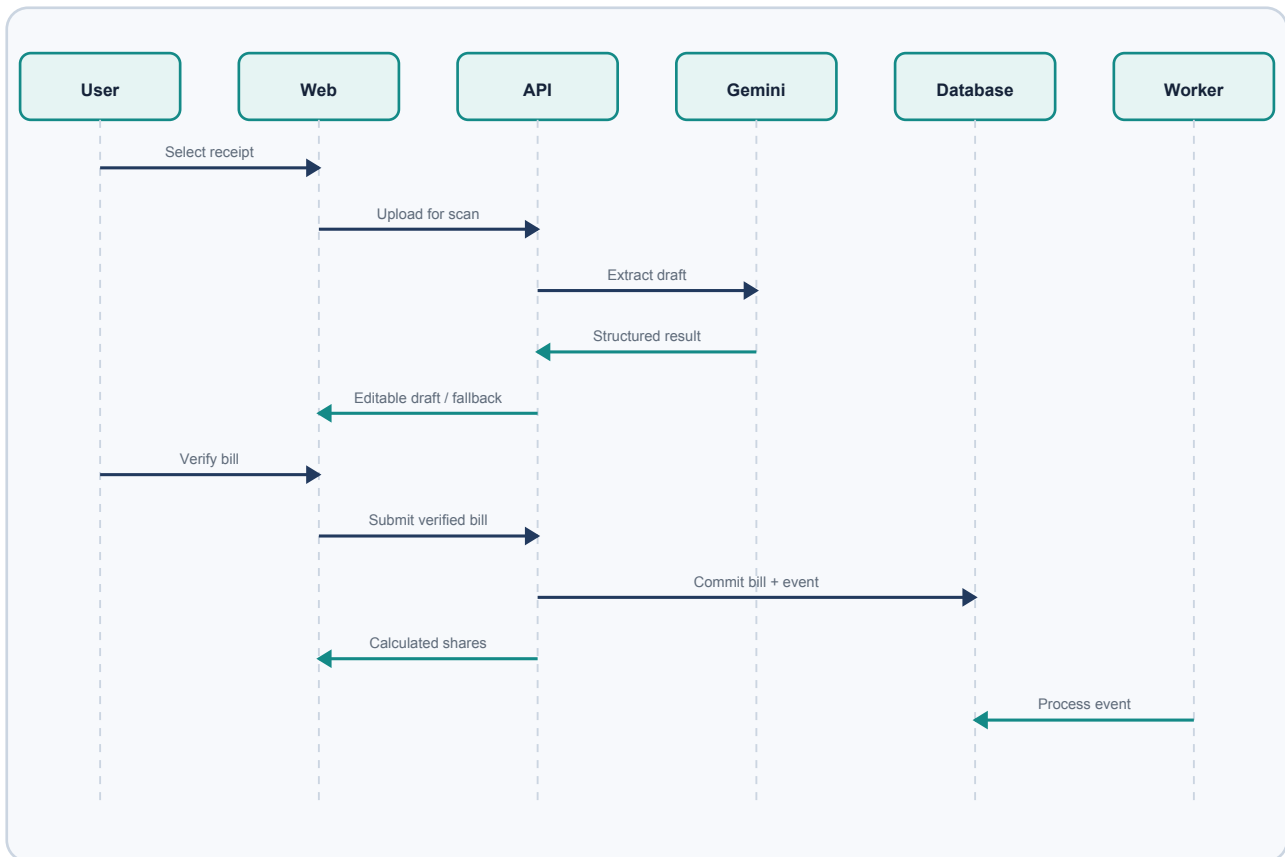


Figure 4. Architecture view

Important baseline gap: the current `/v1/bills/scan` route validates the image but does not apply `authMiddleware.isAuthorized`. Production release requires authentication, ownership/rate checks, request timeout, and a normalized OCR response contract.

7.2 Item allocation and charge distribution

For each item i , the item base is:

```

itemBase[i] = quantity[i] * unitPrice[i]
memberItemShare[i] = itemBase[i] / numberOfAssignedMembers[i]

```

Let `subtotal` be the sum of item bases and `finalTotal` include tax, service fee, tip, and discount. The implemented backend currently uses an adjustment ratio for item-based splitting:

```

adjustmentRatio = finalTotal / subtotal
memberTotal[m] = sum(memberItemShare[i] * adjustmentRatio for items assigned to m)

```

This proportionally distributes both surcharges and discounts according to consumption. Before saving, the server must enforce:

- At least one participant and one payer; both belong to the bill/group.
- Positive item quantity and price; every included item has at least one assignee.

- `subtotal > 0` for an item-based split.
- All monetary values use integer VND (or explicit minor units), never binary floating point as canonical storage.
- `sum(memberTotal) == finalTotal` after rounding. Assign the remainder deterministically (recommended: payer, then stable user ID) and expose it in the breakdown.
- A submitted client total is advisory; the backend recalculates and rejects unexplained mismatches.

Required hardening: the web currently multiplies `quantity * amount`, while the backend adjustment subtotal sums `item.amount` without quantity. Align the API contract to `unitPrice`, calculate `quantity * unitPrice` on the server, and add shared test fixtures before claiming item-based correctness.

7.3 Settle up

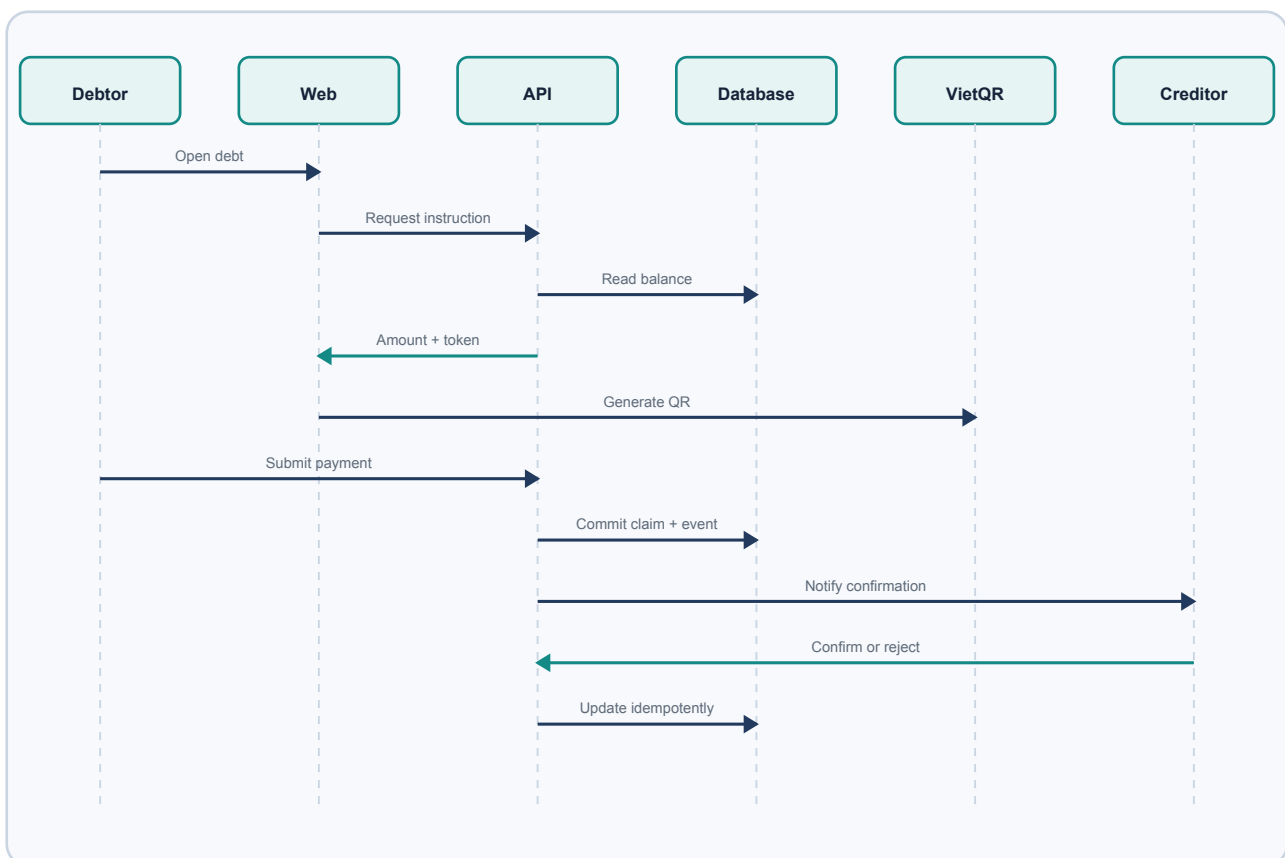


Figure 5. Architecture view

QR generation failure falls back to plain bank name, account number, amount, and transfer content. A QR being displayed or scanned never marks a debt as paid.

8. Data Design

MongoDB documents follow aggregate boundaries; references use `ObjectId`. The detailed field list is maintained in [database.md](#), while the minimum architecture-level model is:

Aggregate/collection	Key fields	Notes
User / <code>users</code>	id, email, name, password hash, verification state, bank code/account, timestamps	Never return password or verification token in normal projections. Bank data is sensitive.
Group / <code>groups</code>	id, creatorId, memberIds, billIds, name, timestamps	Membership controls access to group bills.

Aggregate/collection	Key fields	Notes
Bill / <code>bills</code>	id, creatorId, payerId, participants, items, final total, split method, payment status, deadline	Canonical calculation and settlement aggregate. Embed the immutable calculation breakdown used at creation.
Activity / <code>activities</code>	actor, type, resource, safe details, timestamp	Append-only audit-oriented record; not event sourcing. Do not store secrets or raw receipt content.
Notification / <code>notifications</code>	recipient, actor, type, resource, read state, timestamp	Created idempotently by event handlers and retrieved through the notification API.
Outbox event / <code>outbox_events</code>	eventId, type, aggregateId, payload, status, attempts, availableAt, processedAt	Durable handoff from business transaction to asynchronous notification/activity handlers.
Payment reminder	token hash, debtor, creditor, expiry, usedAt	Token should be single-use, short-lived, and stored hashed.
Payment confirmation	payment reference, payer, recipient, amount, decision, timestamp	Enforce uniqueness/idempotency for repeated callbacks/submissions.

Data consistency

- Business state and its outbox event are committed in one MongoDB transaction. Notification and activity handlers run after commit and do not delay the user-facing request.
- A worker atomically claims an event, applies idempotent handlers keyed by `eventId`, and marks it processed. Transient failures use exponential backoff; exhausted events move to a dead-letter state for operator review.
- Updates use resource filters that include the authorized user/group, not only `_id`.
- Soft-deleted records are excluded consistently. Financial activities and confirmations use retention rules rather than casual deletion.
- Indexes cover user email, bill creator/payer/participants/status/date, group creator/members, and activity timeline. Add unique indexes for external/single-use tokens.

9. Receipt Image Lifecycle and Privacy

The safest MVP policy is **transient processing**:

1. The browser previews the selected file locally.
2. The API accepts only BMP/PNG/JPEG/WebP, checks the real MIME signature, limits size/dimensions/aspect ratio, and resizes oversized input.
3. The image is held in memory only long enough to call Gemini 2.5 Flash. It is not written to MongoDB, application logs, or permanent object storage.
4. Only the user-verified structured fields needed for the bill are saved.
5. Buffers and request references are released after success or failure; provider retention is governed by the provider agreement and disclosed in the privacy notice.

If future product requirements need image retention, use private object storage, encryption, per-bill authorization, short-lived signed URLs, malware/content checks, an explicit retention period, and user deletion controls. Do not embed base64 receipts in MongoDB bill documents.

10. External Integration Contracts and Fallbacks

Dependency	Normal path	Failure policy	Data minimization
Gemini 2.5 Flash	Multimodal request returns a structured receipt draft	Short timeout; at most one bounded retry with jitter for transient errors; return recoverable status; preserve user form; offer manual entry	Send receipt image and extraction prompt only; never send password, bank account, or unrelated group data
VietQR	Render QR from bank code/account, amount, and transfer content	Display copyable payment instructions; allow regeneration; never change payment state	Send only fields required to encode payment instruction
SMTP/Brevo	Send verification, reminder, and payment notifications	Persist in-app notification; retry asynchronously; surface delivery status; do not roll back a valid bill	Email address plus minimum message content
Domain-event outbox	Deliver bill/payment events to notification handlers	Retry with backoff; idempotent handler; dead-letter after the configured attempt limit; alert on event age	Event payload contains identifiers and minimum display fields, not receipt images or secrets
MongoDB	Read/write canonical application state	Fail request safely; no false success; health monitoring, backups, and tested restore	Private network and least-privilege database user

Provider adapters must map errors to stable internal codes such as `OCR_UNAVAILABLE`, `OCR_INVALID_RESPONSE`, and `EMAIL_DELIVERY_DEFERRED`; raw provider responses and credentials must not reach the browser or logs.

11. API Design

The API is versioned under `/v1`. Major resource families currently include users, groups, bills, debts, payment confirmation, activities, history, reports, dashboard, notifications, and assistant functions.

Contract conventions

- JSON requests/responses; consistent `{ data, error, meta }` envelope is the target contract.
- Joi validates shape and limits at the HTTP boundary; domain services validate business invariants.
- Authentication identity comes from the verified JWT, never a trusted `userId` body/path field.
- Authorization checks membership/ownership for every resource read or mutation.
- Pagination is mandatory for list/history/activity endpoints.
- Mutation endpoints accept an idempotency key where retries can duplicate a payment or notification.
- Error responses use stable codes and safe Vietnamese user messages; stack traces remain server-side.
- `/v1/status` is a liveness check; add a readiness check for MongoDB and critical configuration.

High-risk route review before production

Current code has public routes for OCR scanning, some history reads, AI assistant requests, guest creation, opt-out, payment submission/confirmation, and test email utilities. Some public token-based flows are intentional, but each needs purpose-specific signed/hashed tokens, expiry, rate limiting, and strict response minimization. Protect or remove diagnostic `/v1/test/*` routes in production.

12. Security Architecture

Authentication and session

- Keep JWT access tokens in `HttpOnly`, `Secure`, `SameSite` cookies with short expiry; rotate/revoke refresh credentials if refresh tokens are introduced.
- Hash passwords with bcrypt using an approved work factor and never log credentials/tokens.
- Apply login, registration, OCR, AI, email, and token-verification rate limits.
- Require verified identity for financial/group operations unless a narrowly scoped guest flow is explicitly designed.

Authorization

- Derive the actor ID from `req.jwtDecoded`; reject mismatched `creatorId`, `userId`, `payer`, or participant claims.
- Enforce group membership and bill participation in service/repository queries.
- Only the creditor/authorized group member can confirm a payment; confirmation is idempotent and auditable.
- Web route protection is UX only; all enforcement happens in the API.

Application and infrastructure

- Enforce HTTPS, production CORS allow-list, security headers, request/body limits, and CSRF protection for cookie-authenticated mutations.
- Validate actual upload type, not filename; never execute uploaded content.
- Store secrets in deployment secret storage, not source or client environment variables; rotate exposed credentials.
- Redact authorization headers, cookies, bank accounts, receipt images, OCR payloads, and personal data from logs.
- Encrypt backups and database/provider traffic; restrict database network access and credentials.
- Maintain dependency scanning, lockfiles, protected deployment secrets, backup/restore tests, and incident procedures.

13. Reliability, Performance, and Observability

Initial service targets

Area	MVP target
Core API	p95 under 500 ms for ordinary authenticated reads/writes, excluding external providers
Bill calculation	Under 200 ms for up to 100 items and 50 participants
OCR	User-visible progress; timeout within 30 seconds; manual fallback always available
Availability	Core manual bill/settlement records remain usable when OCR, email, event workers, or VietQR fails
Correctness	100% of calculation fixtures preserve <code>sum(shares) == finalTotal</code> ; idempotent payment confirmation
Recovery	Daily backup at minimum; documented and tested restore before production acceptance
Accessibility	Keyboard-operable forms/dialogs, labeled controls, readable errors, and WCAG AA color contrast target

Observability

- Structured logs with request/correlation ID, route, duration, status, internal error code, and provider latency.
- Metrics for request/error rate, p95 latency, MongoDB pool health, OCR success/invalid/timeout rate, outbox queue depth/age, worker retries/dead letters, email delivery, and calculation invariant failures.
- Alerts on elevated 5xx rate, database unavailability, sustained OCR failure, and failed backup.
- Audit activities record relevant actor/action/resource/result without storing sensitive payloads.

14. UX Routes and Architecture Traceability

Route/screen	Workflow responsibility	Main API/data dependency	Failure/fallback
<code>/login, /register</code>	Establish verified identity	Users/auth	Clear validation; retry verification safely
<code>/groups, /groups/:groupId</code>	Create group, add/manage members	Groups, users	Bill creation can still select individual participants
<code>/ocr</code>	Select, validate, and scan receipt	Gemini 2.5 Flash through <code>/v1/bills/scan</code>	Retry or continue to <code>/create</code> manually
<code>/create</code>	Verify/edit OCR, choose payer, assign items, split charges	Bills, users/groups; authoritative calculation on API	Preserve draft and show field-level mismatch
<code>/bills/:billId, /history</code>	Explain bill and audit prior records	Bills/history/activity	Read-only cached shell; safe retry
<code>/debt</code>	Show who owes whom and initiate settlement	Debt/payment services	Plain payment instructions if QR fails
<code>/payment/pay, /payment/confirm</code>	Token-scoped payment submission/confirmation	Payment records and bill status	Expired/used token explanation; never double apply
<code>/dashboard, /reports</code>	Derived spending/debt insights	Bills/report queries	Show partial/empty state without mutating source data

Workflow-to-feature traceability:

User workflow	Architecture capability	Acceptance focus
Create group and members	Identity + Groups modules	Only authorized members see or change the group
Scan/enter receipt	Upload validation + Gemini adapter + manual form	Valid image becomes editable draft; provider failure does not block manual flow
Assign a shared item	Bill aggregate + allocation UI	N assignees receive equal base share unless another explicit rule is selected
Allocate VAT/fees/discount	Calculation service	Proportional distribution is explainable and exact after rounding
Record upfront payer	Bill/payment state	Payer's own share is accounted for; others owe the payer/net creditors
Settle and create QR	Debt/payment module + VietQR	QR matches instruction; QR display alone never marks paid
Track paid/unpaid	Confirmation + activity + notification	Repeated confirmation is idempotent and status is auditable

15. Technical POC and Delivery Plan

The POC must prove the calculation core before depending on OCR or QR.

POC-1: Core calculation (acceptance gate)

Create group -> manually enter bill -> assign items -> calculate shares -> show settlement

Required evidence:

- Automated fixtures for equal, per-person, individual-item, shared-item, surcharge, discount, and rounding-remainder cases.
- Invalid cases: no participant, unassigned item, zero subtotal, payer outside participants, negative value, and mismatched total.
- Server result, UI display, and persisted breakdown agree exactly.

POC-2: OCR-assisted entry

- Validate and resize supported images.
- Map Gemini 2.5 Flash structured output into a strict internal draft schema.
- Demonstrate malformed JSON, missing fields, low-quality image, timeout, and manual-edit fallback.
- Verify that no raw receipt image is persisted or logged.

POC-3: Settlement assistance

- Generate a QR/payment instruction from verified creditor details and amount.
- Demonstrate plain-text fallback and single-use confirmation.
- Confirm that retries cannot increase `amountPaid` twice.

Recommended delivery order

1. Freeze money/item API schema and calculation fixtures.
2. Move all authoritative calculation to the backend and persist the breakdown.
3. Complete resource authorization and protect/remove public diagnostic endpoints.
4. Normalize OCR contract, add timeout/rate limit, and implement transient-image policy.
5. Harden payment confirmation, QR fallback, notification outbox/retry, and observability.
6. Containerize, add CI tests/security checks, and perform restore/load/usability tests.

16. Risks, Constraints, and Mitigations

Risk/constraint	Impact	Mitigation / decision trigger
OCR is inaccurate on blurred or unusual Vietnamese receipts	Wrong draft items/totals	Human confirmation, field warnings, manual entry, OCR quality metrics; never auto-settle from OCR
Frontend/backend calculation drift	Disputed balances	One authoritative backend library/contract, integer money, shared fixtures, persisted explanation
External provider outage or quota	Scan/QR/email unavailable	Timeouts, bounded retries, manual/plain fallback, quotas and alerts
Unauthorized cross-user access through path/body IDs	Financial/privacy breach	Actor from JWT, resource-scoped queries, authorization tests for every endpoint
Public token reuse or guessing	Fraudulent settlement changes	High-entropy hashed token, expiry, single use, rate limit, idempotent update, audit
Event worker outage or growing outbox backlog	Delayed notifications	Independent worker health check, retry/backoff, queue-age alert, dead-letter review, and safe replay
Single API process/VPS	Limited fault tolerance	Health checks, PM2 restart, backup/restore, reverse proxy; add replicas only after measured need
MongoDB multi-document consistency	Partial activity/notification state	Transactions where needed; outbox and idempotent consumers; bill remains canonical
Premature microservice extraction	Delivery and operating overhead	Keep module boundaries; extract only when independent scaling/ownership or reliability evidence exists